

机器终身学习（二）：缓解灾难性遗忘

Selective Synaptic Plasticity, GEM, Neural Resource Allocation, Memory Replay, and Curriculum



基于李宏毅老师《机器学习 2025》课程内容整理

2026 年 6 月 9 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 36 分 39 秒

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=54>

目录

1 缓解遗忘的三类研究方向	3
1.1 本章小结	3
2 为什么会发生 Catastrophic Forgetting	4
2.1 本章小结	4
3 Selective Synaptic Plasticity: 保护重要参数	5
3.1 保护强度的两端：遗忘与僵化	5
3.2 EWC 与重要性估计	7
3.3 本章小结	9
4 Gradient Episodic Memory: 重定向当前梯度	9
4.1 本章小结	10
5 Additional Neural Resource Allocation: 给新任务分配资源	10
5.1 Progressive Neural Networks	10
5.2 PackNet、CPG 与参数复用	11
5.3 本章小结	12
6 Memory Replay: 让旧任务重新出现	12
6.1 Adding New Classes: 类别增量学习	13
6.2 Curriculum Learning: 学习顺序也会影响遗忘	14
6.3 本章小结	15
7 总结与延伸	15
7.1 本章小结	16

1 缓解遗忘的三类研究方向

P53 说明了 catastrophic forgetting: 模型依序学习任务时, 后续任务会把旧任务能力冲掉。P54 开始讨论解决方向。课程先给出三类方法:

- **Selective Synaptic Plasticity:** 选择性地保护旧任务重要参数, 只让不重要参数更自由地变化。
- **Additional Neural Resource Allocation:** 为新任务分配额外网络资源, 避免所有任务抢同一组参数。
- **Memory Replay:** 通过保留、生成或回放旧任务数据, 让新任务训练时仍看见旧任务。

课程首焦

Research Directions

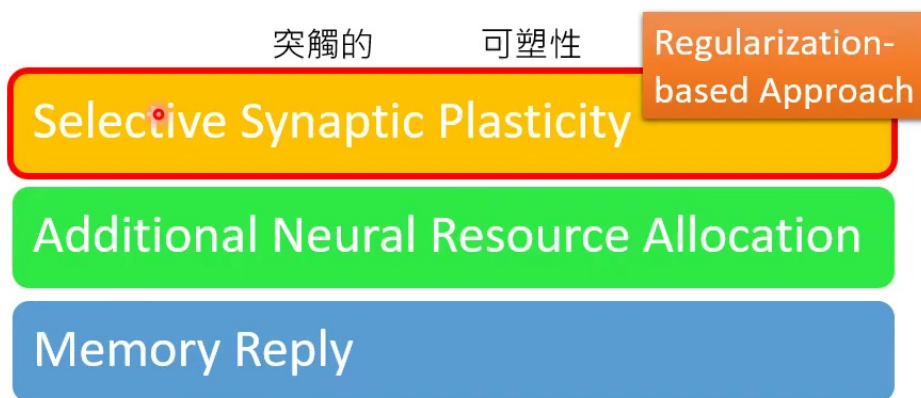


图 1: 缓解 catastrophic forgetting 的三大方向: selective synaptic plasticity、additional neural resource allocation 与 memory replay; 课程首先聚焦 regularization-based approach。¹

这三类方法对应了遗忘问题的三个切入点。Selective synaptic plasticity 直接约束参数更新; additional resources 改变模型容量和任务分配方式; memory replay 则改变训练数据流, 让模型在学新任务时仍接触旧任务信号。

三类方法的核心差异

正则化方法通常不保存大量旧数据, 代价低但保护能力有限; 资源分配方法能显式隔离任务, 但模型会增长或可用参数减少; replay 方法最接近 multi-task training, 但需要存储旧样本或训练生成模型。

1.1 本章小结

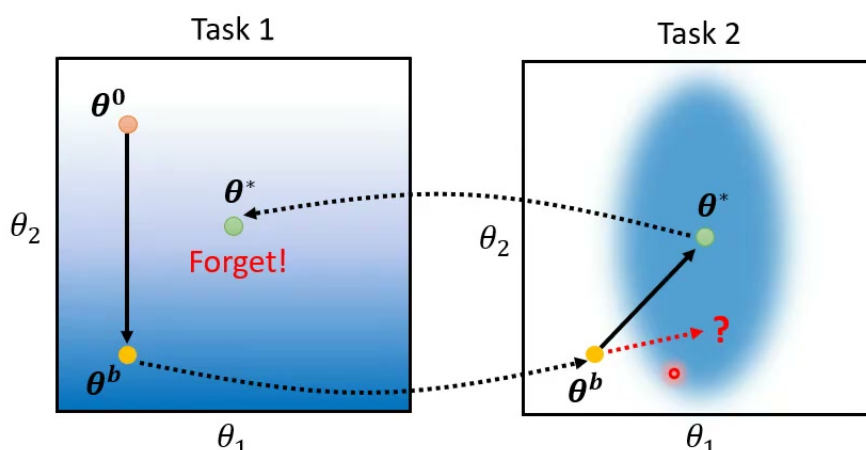
P54 的方法地图围绕三个问题展开: 哪些参数该被保护、是否给新任务新资源、以及如何让旧任务数据继续参与训练。后续每类方法都在 stability 与 plasticity 之间做取舍。

¹视频画面时间区间: 00:01:30-00:01:45。

2 为什么会发生 Catastrophic Forgetting

课程用二维参数空间解释遗忘。设模型参数为 (θ_1, θ_2) , task 1 与 task 2 有不同 error surface, 颜色越深表示 loss 越小。模型先从 θ^0 训练到 task 1 的好解 θ^b ; 接着训练 task 2 时, 梯度会把参数推向 task 2 的低 loss 区域 θ^* 。

Why Catastrophic Forgetting?



The error surfaces of tasks 1 & 2.
(darker = smaller loss)

图 2: 两个任务的 error surface 不同: 从旧任务解 θ^b 往 task 2 低 loss 区移动时, 参数可能离开 task 1 的低 loss 区, 导致 task 1 被遗忘。²

遗忘的关键不在于 task 2 本身有多难，而在于 task 2 的好解和 task 1 的好解不一定重合。若优化器只看当前任务 loss，它会沿着降低 task 2 loss 的方向更新；这个方向可能同时让 task 1 loss 上升。越多任务共享同一组参数，这种冲突越容易出现。

从这个图也能看出几种可能策略：

- 保护 task 1 重要方向，不让参数离开旧任务低 loss 区太远。
- 找一个对 task 2 有利、但不增加 task 1 loss 的更新方向。
- 给 task 2 新参数，减少它和 task 1 对旧参数的争抢。
- 在训练 task 2 时继续用 task 1 数据提醒模型。

2.1 本章小结

Catastrophic forgetting 可以看成参数空间中的任务冲突：当前任务梯度把参数推向新任务低 loss 区，却可能推离旧任务低 loss 区。P54 的方法本质上都在限制、重定向或补偿这种参数移动。

²视频画面时间区间: 00:05:15—00:05:30。

3 Selective Synaptic Plasticity: 保护重要参数

Selective Synaptic Plasticity 的基本思想是：模型中并非所有参数对旧任务都同等重要。某些参数如果改变，旧任务性能会大幅下降；另一些参数即使改变，旧任务也不太受影响。因此，学习新任务时应更多修改不重要参数，尽量保护重要参数。

课程给出一个正则化形式：

$$L'(\theta) = L(\theta) + \lambda \sum_i b_i (\theta_i - \theta_i^b)^2.$$

其中 θ^b 是从旧任务学到的参数， b_i 是第 i 个参数的 guard 或 importance， λ 控制保护强度。若 b_i 大， θ_i 离开 θ_i^b 会受到更大惩罚；若 b_i 小，该参数更容易被新任务更新。

3. Selective Synaptic Plasticity

Selective Synaptic Plasticity

Basic Idea: Some parameters in the model are important to the previous tasks. Only change the unimportant parameters.

θ^b is the model learned from the previous tasks.

Each parameter θ_i^b has a “guard” b_i

$$L'(\theta) = L(\theta) + \lambda \sum_i b_i (\theta_i - \theta_i^b)^2$$

图 3: Selective synaptic plasticity 的正则化目标：当前任务 loss 加上参数偏离旧任务参数的惩罚，每个参数的惩罚由重要性 b_i 加权。³

这个式子可以看成从普通 L2 regularization 扩展而来。普通 L2 往往对所有参数同等惩罚；selective synaptic plasticity 则让每个参数有不同保护强度。真正困难的是估计 b_i ：哪些参数对旧任务重要？重要到什么程度？

3.1 保护强度的两端：遗忘与僵化

如果 $b_i = 0$ ，第 i 个参数没有任何约束，学习新任务时可以随意改变。这提升 plasticity，但可能造成 catastrophic forgetting。若 $b_i = \infty$ ，参数永远不能离开旧值 θ_i^b ，旧任务被保护，但新任务可能学不动，这叫 intransigence。

³视频画面时间区间：00:08:00–00:08:15。

Selective Synaptic Plasticity

Basic Idea: Some parameters in the model are important to the previous tasks. Only change the unimportant parameters.

θ^b is the model learned from the previous tasks.

Each parameter θ_i^b has a “guard” b_i

θ should be close to θ^b in certain directions.

$$L'(\theta) = L(\theta) + \lambda \sum_i b_i (\theta_i - \theta_i^b)^2$$

If $b_i = 0$, there is no constraint on θ_i ➡ Catastrophic Forgetting

If $b_i = \infty$, θ_i would always be equal to θ_i^b ➡ Intransigence

图 4: 参数重要性 b_i 的两端: $b_i = 0$ 时没有约束, 容易遗忘; $b_i = \infty$ 时参数不能改变, 模型会过度僵化, 难以学习新任务。⁴

因此 continual learning 的目标不是把所有旧参数冻结, 而是估计 “哪些方向必须靠近旧解, 哪些方向可以让给新任务”。课程图中用二维参数说明: 若 b_1 小而 b_2 大, 模型可以主要改变 θ_1 , 但尽量不改变 θ_2 , 从而在 task 2 中寻找不破坏 task 1 的解。

⁴视频画面时间区间: 00:11:30–00:11:45。

Selective Synaptic Plasticity

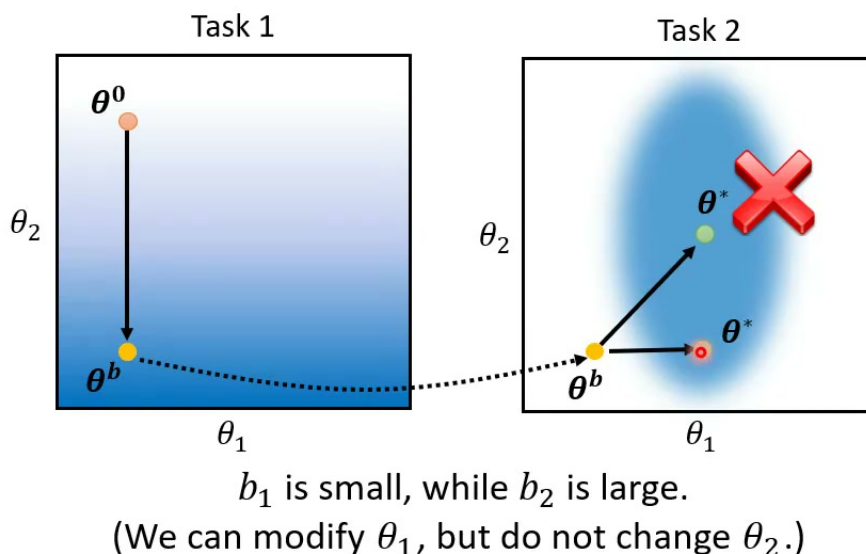


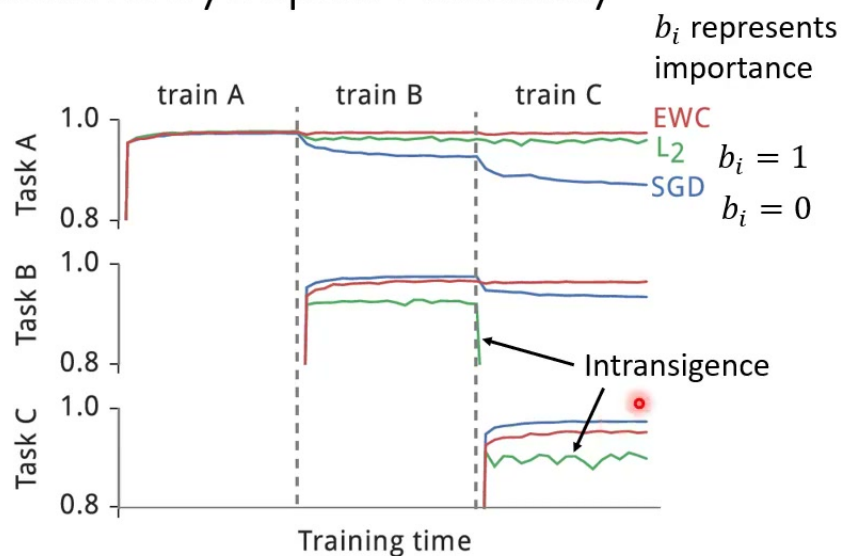
图 5: 选择性保护的几何直觉: 当 b_1 小、 b_2 大时, 可以沿 θ_1 方向调整模型, 但不要大幅改变 θ_2 ; 这限制了新任务更新的可行方向。⁵

3.2 EWC 与重要性估计

Elastic Weight Consolidation (EWC) 是代表性方法之一。它用 Fisher information 估计每个参数对旧任务 loss 的敏感程度: 若某参数附近的 loss 曲率大, 改变它会显著破坏旧任务, 因此 b_i 应该大。课程图中比较 EWC、普通 L2 和 SGD, 在 permuted MNIST 的 task A、B、C 上, EWC 更能维持旧任务表现。

⁵视频画面时间区间: 00:14:45-00:15:00。

Selective Synaptic Plasticity



MNIST permutation, from the original EWC paper

图 6: EWC、L2 与 SGD 的比较: 在连续训练 task A、B、C 时, EWC 通过参数重要性保护旧任务, 通常比普通 SGD 更不易遗忘; 但过度保护会带来 intransigence。⁶

课程也列出一组相关方法: EWC、Synaptic Intelligence (SI)、Memory Aware Synapses (MAS)、RWalk、Sliced Cramer Preservation (SCP)。它们的共同点是: 不直接保存大量旧数据, 而是用某种方式估计旧任务对参数的约束, 再在新任务训练中加入正则项。

⁶视频画面时间区间: 00:18:30-00:18:45。

Selective Synaptic Plasticity

- Elastic Weight Consolidation (EWC)
 - <https://arxiv.org/abs/1612.00796>
- Synaptic Intelligence (SI)
 - <https://arxiv.org/abs/1703.04200>
- Memory Aware Synapses (MAS)
 - <https://arxiv.org/abs/1711.09601>
- RWalk
 - <https://arxiv.org/abs/1801.10112>
- Sliced Cramer Preservation (SCP)
 - <https://openreview.net/forum?id=BJge3TNKwH>

图 7: Selective synaptic plasticity 的相关方法: EWC、SI、MAS、RWalk 与 SCP 等, 核心都在于估计并保护旧任务重要参数。⁷

正则化保护无法凭空创造容量

如果新旧任务确实需要冲突很大的参数配置, 单靠正则项只能在遗忘和僵化之间折中。保护太弱会忘, 保护太强会学不动; 保护参数也不能解决旧数据分布完全不可恢复的问题。

3.3 本章小结

Selective synaptic plasticity 用重要性加权的正则项保护旧任务参数。它的优点是额外存储较少、实现相对直接; 缺点是需要估计参数重要性, 并且在任务冲突强时可能面临遗忘与 intransigence 的两难。

4 Gradient Episodic Memory: 重定向当前梯度

GEM (Gradient Episodic Memory) 从梯度角度处理遗忘。它保留一小部分旧任务数据, 训练当前任务时也计算旧任务上的梯度。若当前任务梯度会让旧任务 loss 上升, 就把当前梯度投影到一个不伤害旧任务的方向。

设当前任务负梯度为 g , 旧任务 memory 上的负梯度为 g^b 。GEM 希望找到一个尽量接近 g 的更新方向 g' , 同时满足

$$g'^{\top} g^b \geq 0.$$

这个约束表示: 新更新方向与旧任务下降方向不冲突, 至少在一阶近似下不会增加旧任务 loss。更一般地, 对每个旧任务 k 都有约束 $g'^{\top} g_k \geq 0$ 。

$$\min_{g'} \|g' - g\|_2^2 \quad \text{s.t.} \quad g'^{\top} g_k \geq 0, \forall k < t.$$

⁷视频画面时间区间: 00:22:15–00:22:30。

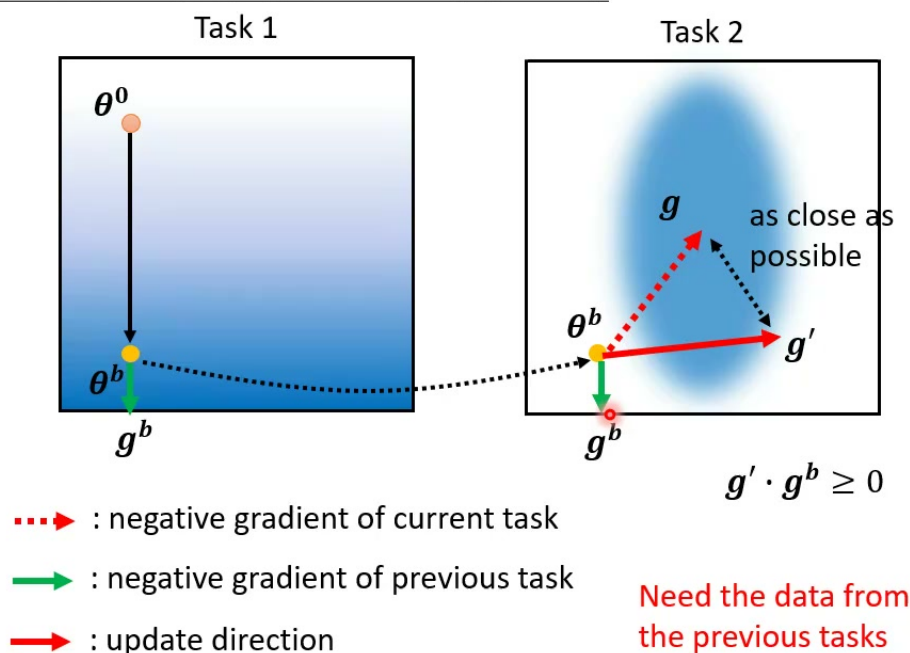


图 8: GEM 的几何图: 红色虚线是当前任务负梯度, 绿色是旧任务负梯度, 红色实线 g' 是投影后的更新方向; 约束 $g' \cdot g^b \geq 0$ 需要旧任务数据来估计。⁸

GEM 和 EWC 的差别在于, EWC 用参数重要性近似旧任务约束, GEM 则直接使用旧任务样本估计“当前更新会不会伤害旧任务”。因此 GEM 更接近 replay 方法, 但它不是简单把旧样本混进 batch, 而是把旧样本用于约束梯度方向。

GEM 的关键代价

GEM 需要保存旧任务样本或代表性 memory。memory 越大, 对旧任务约束越可靠, 但存储和计算越高; memory 太小, 则旧任务梯度可能不能代表完整旧分布。

4.1 本章小结

GEM 把 continual learning 写成一个带约束的梯度更新问题: 尽量沿当前任务下降方向走, 但不能让旧任务 loss 上升。它比纯正则化更直接利用旧任务数据, 也因此付出 memory 与额外梯度计算成本。

5 Additional Neural Resource Allocation: 给新任务分配资源

第二大类方法不再只限制旧参数, 而是为新任务分配额外神经资源。直觉是: 如果所有任务都挤在同一组参数里, 冲突不可避免; 若新任务有自己的参数子空间, 旧任务参数就不必被大幅修改。

5.1 Progressive Neural Networks

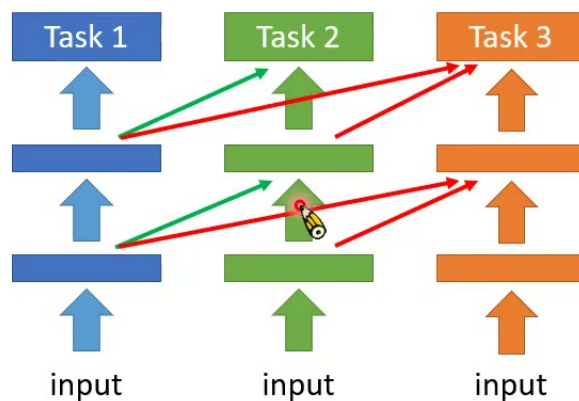
Progressive Neural Networks 为每个新任务增加一个新 column。旧任务 column 被冻结, 避免遗忘; 新任务 column 可以通过 lateral connections 读取旧任务表示, 从而迁移旧知识。这样每个任

⁸视频画面时间区间: 00:26:00-00:26:15。

务都有独立路径，又能利用过去学到的特征。



Progressive Neural Networks



<https://arxiv.org/abs/1606.04671>

图 9: Progressive Neural Networks: 每个任务新增一列网络，旧列冻结，新列通过横向连接使用旧任务表示；这样避免遗忘，但模型规模随任务数增长。⁹

优点是遗忘少，因为旧任务参数不再被更新；缺点也明显：任务越多，模型越大。如果任务流很长，参数量和推理成本都会线性增长。它适合任务数量有限且任务边界明确的场景。

5.2 PackNet、CPG 与参数复用

PackNet 和 Compacting, Picking, and Growing (CPG) 试图更节省地分配参数。PackNet 的思路是：先为一个任务训练网络，再剪枝释放一部分不重要参数；保留并冻结对旧任务重要的参数，空出来的参数给新任务使用。CPG 则在 compact、pick、grow 之间循环，必要时扩展模型。

⁹视频画面时间区间：00:28:30-00:28:45。

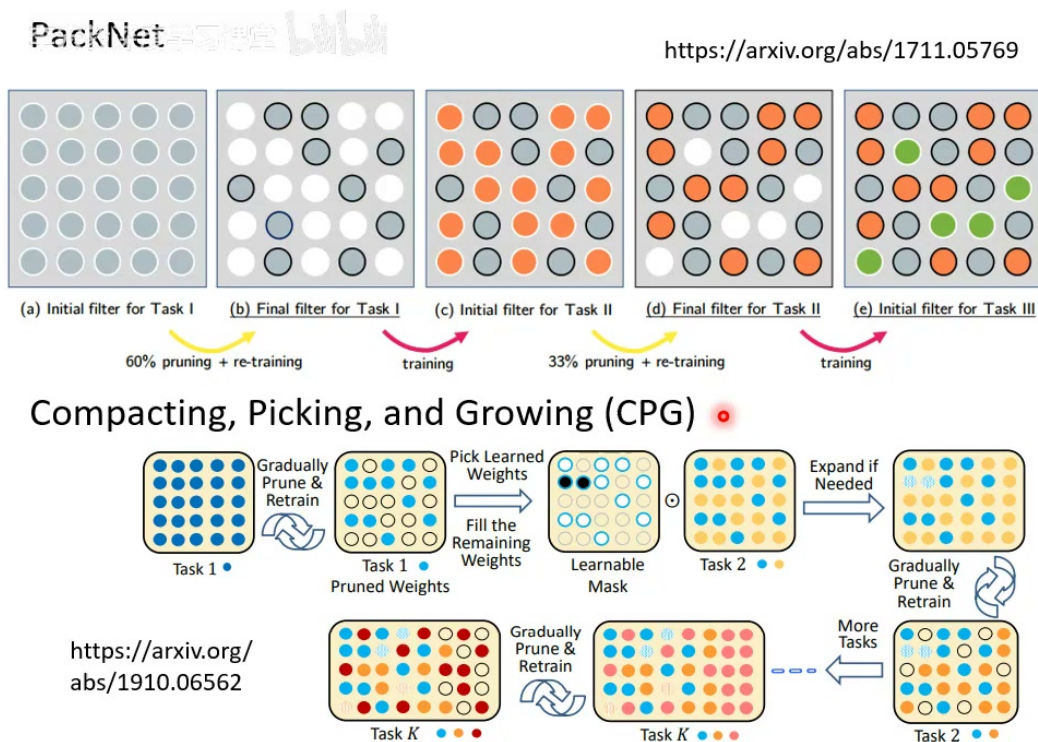


图 10: PackNet 与 CPG 的参数分配思路：通过 pruning、mask、free weights 与必要扩展，为不同任务保留或分配参数子空间。¹⁰

这类方法位于“完全冻结旧模型”和“所有任务共享全部参数”之间。它们让任务共享一部分网络，同时为关键任务保留专属参数。挑战在于任务顺序、剪枝比例、mask 管理和容量耗尽：如果早期任务占用太多参数，后续任务可能没有足够空间。

5.3 本章小结

Additional neural resource allocation 用结构隔离缓解遗忘。Progressive networks 通过新增 column 避免旧任务被改写；PackNet/CPG 则通过剪枝、冻结、复用和扩展来管理参数。它们可靠但资源成本较高。

6 Memory Replay：让旧任务重新出现

第三大方向是 replay。最朴素的 replay 是保存旧任务样本，在训练新任务时和新任务数据一起训练，近似 multi-task training。但保存原始旧数据可能很贵，也可能涉及隐私。课程重点展示了 generative replay：用生成模型产生旧任务 pseudo-data。

¹⁰ 视频画面时间区间：00:30:30–00:30:45。

Generating Data

<https://arxiv.org/abs/1705.08690>
<https://arxiv.org/abs/1711.10563>
<https://arxiv.org/abs/1909.03329>

- Generating pseudo-data using generative model for previous tasks

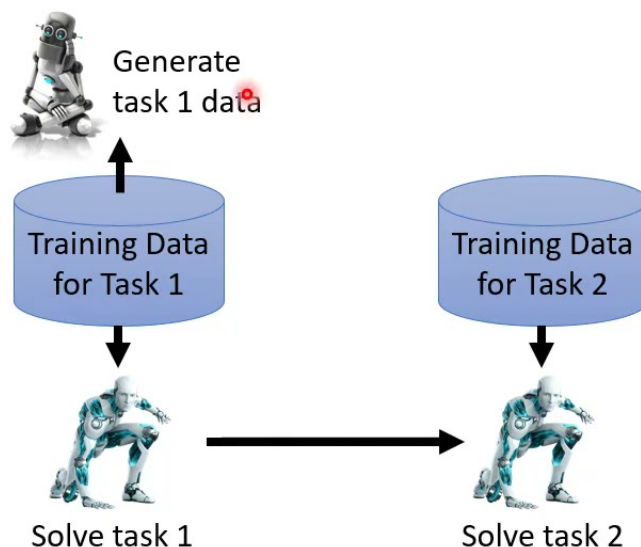


图 11: Generative replay: 为旧任务训练生成模型, 用它产生 pseudo-data; 学习新任务时, 把生成的旧任务样本与新任务数据一起训练, 近似 multi-task learning。¹¹

Generative replay 的关键好处是减少原始数据存储。只要生成模型能保留旧任务分布, 就可以在新任务训练时不断“回放”旧任务。但这也引入新问题: 生成模型本身也可能忘记, 生成质量不足会造成偏差, 且生成模型的训练和存储也有成本。

6.1 Adding New Classes: 类别增量学习

课程也提到 adding new classes 的场景。新任务不是换一个完全不同任务, 而是给分类器增加新类别。例如原本只会分旧类别, 现在要加入新类别; 模型最终需要在所有类别之间判断, 而不是只在当前任务内判断。相关方法包括 Learning without Forgetting (LwF) 和 iCaRL。

¹¹ 视频画面时间区间: 00:31:45–00:32:00。

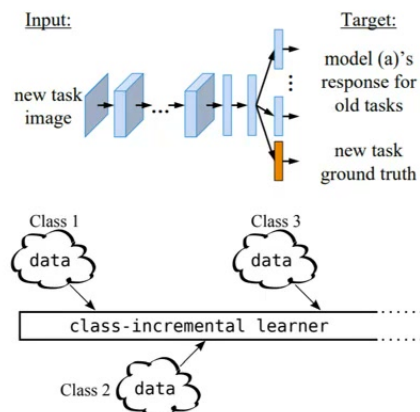
Adding new classes

Learning without forgetting (LwF)

<https://arxiv.org/abs/1606.09282>

iCaRL: Incremental Classifier and Representation Learning

<https://arxiv.org/abs/1611.07725>



Three scenarios for continual learning

<https://arxiv.org/abs/1904.07734>

图 12: Adding new classes 的代表方法: LwF 使用旧模型对旧任务的响应作为目标, iCaRL 关注 class-incremental learner 和表示学习。¹²

LwF 的思想和 knowledge distillation 有关联: 新任务训练时, 让新模型同时匹配旧模型对旧类别的输出, 从而保留旧类别知识。iCaRL 则通常维护少量 exemplar, 并学习适合类别增量的 representation。Class-incremental learning 往往比 task-incremental 更难, 因为测试时模型不一定知道输入属于哪个任务。

6.2 Curriculum Learning: 学习顺序也会影响遗忘

最后课程回到学习顺序。两个任务若按 task 1 $\rightarrow$ task 2 学习, 可能出现 task 1 遗忘; 换成 task 2 $\rightarrow$ task 1, 结果可能不同。Curriculum learning 问的是: 是否存在更合适的任务顺序, 让模型更少遗忘、更容易迁移?

¹²视频画面时间区间: 00:33:45–00:34:00。

Curriculum Learning : what is the proper learning order?

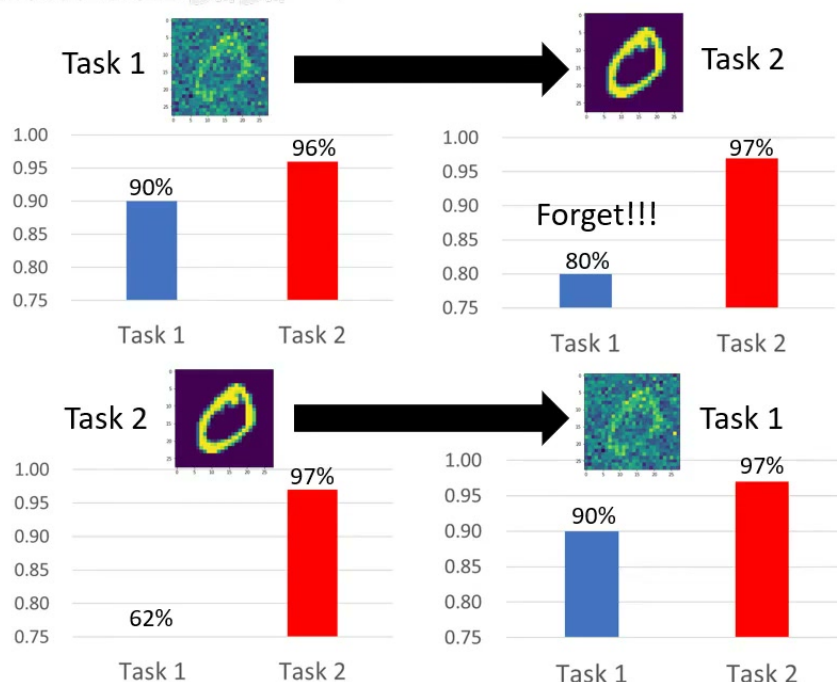


图 13: Curriculum learning 的问题: 不同任务顺序可能导致不同遗忘与迁移结果, 因此需要思考 proper learning order。¹³

这说明 continual learning 不只是算法问题, 也是训练流程问题。任务顺序、任务相似度、数据平衡、replay 频率、模型容量分配, 都会影响最终遗忘程度。

6.3 本章小结

Replay 通过让旧任务信号重新参与训练来缓解遗忘, 可以保存真实样本, 也可以生成 pseudo-data。新类别学习和 curriculum learning 进一步说明, continual learning 的难点不仅在参数保护, 也在任务定义、类别空间和学习顺序。

7 总结与延伸

P54 把 catastrophic forgetting 的解决方向整理成方法地图:

- **Regularization-based:** 用参数重要性保护旧任务, 例如 EWC、SI、MAS。
- **Gradient-constrained:** 用旧任务 memory 约束当前梯度, 例如 GEM。
- **Resource allocation:** 通过新增、冻结、剪枝、mask 或扩展为不同任务分配参数。
- **Replay-based:** 保存或生成旧任务数据, 让训练过程接近 multi-task learning。
- **Curriculum-aware:** 调整任务顺序, 利用任务之间的迁移关系减少遗忘。

这些方法没有一个单独完美。正则化省存储但可能保护不足; GEM 和 replay 更直接但需要旧数据或 memory; 资源分配稳定但模型会增长; 生成式 replay 省原始数据但依赖生成质量; curriculum 有帮助但不总能控制任务到达顺序。

¹³视频画面时间区间: 00:36:30-00:36:38。

Continual learning 的本质 trade-off

旧任务保护越强，新任务学习越可能受限；新任务学习越自由，旧任务越容易被破坏。所有方法都在 stability、plasticity、memory、compute、model size 与 task transfer 之间做权衡。

从 P53 到 P54，课程完成了一个闭环：先定义灾难性遗忘和评价指标，再给出缓解方法。后续真正设计 continual learning 系统时，应先明确约束：是否允许保存旧数据、任务边界是否已知、类别空间是否增长、模型是否可扩展、旧任务表现要保护到什么程度。方法选择必须服从这些约束，而不是只看单一 benchmark 分数。

7.1 本章小结

P54 的核心是方法地图：保护重要参数、约束梯度、分配新资源、回放旧数据、调整学习顺序。它们从不同侧面缓解 catastrophic forgetting，但都无法绕开稳定性与可塑性的根本张力。